Sztuczna inteligencja i inżynieria wiedzy - Lista 1

Zofia Różańska, 280526

Zadanie 1

Wstęp

Pierwszym krokiem do wykonania zadania było załadowanie i przetworzenie danych z plików GTFS. W tym celu stworzyłam klasę `GTFSLoader`. Metoda `load_all(target_date)` wczytuje pliki calendar_dates.txt, calendar.txt, routes.txt, stop_times.txt, stops.txt i trips.txt do `DataFrame` z biblioteki `Pandas`, a nastepnie przetwarza je filtrując dane na podstawie podanej daty `target_date` oraz zapewniając, że każdy `stop` ma swojego `parent_station`.

Po wstępnym przetworzeniu danych, należało zbudować graf modelujący sieć kolejową. W tym celu stworzyłam klasę `GraphBuilder`, która przyjmuje przetworzone ramki danych i buduje graf w postaci słownika, gdzie kluczem jest id `parent_station`, a wartością jest lista krawędzi reprezentujących kolejne przejazdy między stacjami połączonymi kursami pociągów w danym dniu (`Edge`).

Do przechowywania szczegółowych informacji o stacjach stworzyłam również klasę `Stop`. `GraphBuilder.get_metadata()` zwraca słownik, gdzie kluczem jest id `parent_station`, a wartością jest obiekt `Stop` zawierający nazwę stacji i resztę dostępnych o niej informacji.

In [ ]:
```
class Stop:
    def __init__(self, stop_id, stop_name, lat=None, lon=None, lines=None):
        self.stop_id = stop_id
        self.stop_name = stop_name
        self.lat = lat
        self.lon = lon
        self.lines = lines if lines is not None else set()


class Edge:
    def __init__(self, to_stop, departure, arrival, trip_id, route_name):
        self.to_stop = to_stop      # parent_station id
        self.departure = departure  # seconds from midnight
        self.arrival = arrival      # seconds from midnight
        self.trip_id = trip_id
        self.route_name = route_name

    @property
    def duration(self):
        return self.arrival - self.departure
```

```
class GraphBuilder:
    def __init__(self, loader):
        pass

    def build_graph(self):
        pass

    def get_metadata(self):
        pass
```

Wyszukiwanie najszybszego połączenia między stacjami za pomocą algorytmu Dijkstry

Aby znaleźć najlepsze połączenie ze stacji A do stacji B na podstawie kryterium czasu przejazdu, zaimplementowałam klasę `Dijkstra`, która operuje na grafie zbudowanym przez `GraphBuilder`. Algorytm Dijkstry zaimplementowałam z wykorzystaniem kolejki priorytetowej (min-heap) z biblioteki `heapq` do efektywnego znajdowania najtańszej ścieżki.

Implementując algorytm, wiedzę czerpałam z tego źródła. Mój algorytm bazuje na kodzie źródłowym, jednak musiał zostać dostosowany do specyfiki problemu. W klasycznym algorytmie Dijkstry, krawędzie są poprostu nieujemnymi wagami. Jednak w tym przypadku cały stan grafu jest zależny od czasu. Aby znaleźć najlepsze połączenie kolejowe, należało przede wszystkim uwzględnić czasy odjazdów i przyjazdów pociągów, a także czas potrzebny na przesiadkę. Algorytm w kazdej iteracji wybiera z kolejki priorytetowej wierzchołek o najniższym koszcie (stację o najszybszym czasie dotarcia) i rozważa wszystkie połączenia wychodzące z tej stacji - dla każdego połączenia sprawdza, czy można nim dojechać do sąsiedniej stacji szybciej niż dotychczas znany czas dotarcia do tej stacji. Jeśli tak, aktualizuje czas dotarcia i dodaje sąsiednią stację do kolejki priorytetowej. Użycie kolejki priorytetowej pozwala na efektywne znajdowanie najtańszej ścieżki w grafie, nawet gdy graf jest duży i złożony.

In [ ]:
```
import heapq

class Dijkstra:

    def __init__(self, builder):
        self.graph = builder.build_graph()

    def shortest_path(self, start_station, goal_station, start_time, transfer_bu
        if start_station not in self.graph or goal_station not in self.graph:
            return None, float("inf")

        priority_queue = []

        best_costs = {}
        parent = {}
        parent_edge = {}
        entry_count = 0  # tie-breaker for the priority queue

        initial_cost = start_time
```

```
        # elements in queue: (cost, tie_breaker, current_time, station_id, trip_
        heapq.heappush(priority_queue, (initial_cost, entry_count, start_time, s
        best_costs[start_station] = initial_cost

        while priority_queue:
            cost, _, current_time, u, current_trip = heapq.heappop(priority_queu

            if u == goal_station:
                return self._reconstruct_path(parent, parent_edge, start_station

            for edge in self.graph.get(u, []):
                is_transfer = 1 if current_trip is not None and current_trip !=

                required_departure = current_time
                if is_transfer:
                    required_departure += transfer_buffer

                if edge.departure < required_departure:
                    continue

                new_cost = edge.arrival

                v = edge.to_stop
                if v not in best_costs or new_cost < best_costs[v]:
                    best_costs[v] = new_cost
                    parent[v] = u
                    parent_edge[v] = edge
                    entry_count += 1
                    heapq.heappush(priority_queue, (new_cost, entry_count, edge.

        return None, float("inf")

    def _reconstruct_path(self, parent, parent_edge, start_station, goal_station
        path = []
        current = goal_station
        while current in parent_edge:
            path.append((current, parent_edge[current]))
            current = parent[current]

        path.append((start_station, None))
        path.reverse()
        return path
```

Wyszukiwanie optymalnego połączenia za pomocą algorytmu A*

Aby znaleźć najlepsze połączenie ze stacji A do stacji B, zaimplementowałam klasę `AStar`. Algorytm ten stanowi rozszerzenie Dijkstry o funkcję heurystyczną $h(n)$, która pozwala nadać poszukiwaniom kierunek i skupić się na wierzchołkach przybliżających nas do celu. W mojej implementacji algorytm minimalizuje funkcję kosztu $f(n) = g(n) + h(n)$, gdzie $g(n)$ to rzeczywisty koszt dotarcia do danej stacji, a $h(n)$ to szacowany koszt dotarcia ze stacji do celu.

Tak samo jak przy implementacji algorytmu Dijkstry, implementacja algorytmu A* musiała zostać dostosowana do specyfiki problemu. W klasycznym algorytmie A*, krawędzie są poprostu nieujemnymi wagami. Jednak w tym przypadku cały stan grafu jest zależny od

czasu. Aby znaleźć najlepsze połączenie kolejowe, należało przede wszystkim uwzględnić czasy odjazdów i przyjazdów pociągów, a także czas potrzebny na przesiadkę.

Podobnie jak w przypadku Dijkstry, wykorzystałam kolejkę priorytetową `heapq` do zarządzania zbiorem wierzchołków do sprawdzenia (`open_list`). Kluczową różnicą jest jednak kryterium sortowania w kolejce - zamiast samego czasu dotarcia, algorytm bierze pod uwagę sumę kosztu rzeczywistego i heurystyki. Dzięki temu rozwiązanie jest znajdowane znacznie szybciej, ponieważ algorytm w pierwszej kolejności sprawdza pociągi jadące w stronę stacji docelowej, odkładając na później te, które fizycznie oddalają nas od celu.

Modyfikacje poprawiające wydajność i jakość rozwiązania

W trakcie implementacji wprowadziłam kilka modyfikacji, które pozwoliły na poprawienie wydajności i jakości zwracanych rozwiązań.

- **Rozszerzenie definicji stanu:** W klasycznym algorytmie stanem jest tylko wierzchołek (stacja). W moim rozwiązaniu stanem jest krotka `(station_id, trip_id)` . Wprowadzona zmiana sprawiła, że algorytm wybiera połączenia z mniejszą liczbą przesiadek nawet przy kryterium optymalizacji czasu. Bez tej zmiany, wynikiem były trasy, w których istniały bezcelowe przesiadki.

- **Heurystyka dla czasu:** Dla kryterium czasu przejazdu zastosowałam heurystykę opartą na przewidywanym, idealnym czasie przejazdu. Jest on liczony jako iloraz odległości obliczanej wzorem haversine podzielonej przez maksymalną prędkość pociągu (na Polskich torach standardowo 160 km/h). Dzięki temu heurystyka nigdy nie "kłamie", że dojedziemy gdzieś szybciej niż jest to fizycznie możliwe, co gwarantuje znalezienie najszybszego połączenia.

- **Heurystyka dla przesiadek:** W trybie minimalizacji przesiadek zaimplementowałam sprawdzanie spójności linii. Algorytm sprawdza, czy stacja bieżąca i docelowa obsługują te same numery linii. Jeśli nie, heurystyka zwraca wartość 1, informując algorytm, że pasażera czeka co najmniej jedna przesiadka. Pozwala to na znacznie szybsze odrzucanie ścieżek prowadzących w "ślepe uliczki" sieci kolejowej.

- **Mechanizm Re-openingu:** Zaimplementowałam mechanizm usuwania wierzchołków z listy zamkniętej (`closed_list`) i przenoszenia ich z powrotem do `open_list` , jeśli znaleziona zostanie nowa, lepiej oceniona droga do danego stanu. Zapewnia to znalezienie prawdziwego optimum.

- **Złożony koszt:** Zarówno w $g(n)$ jak i $f(n)$ wykorzystałam krotki kosztów. W trybie czasu priorytetem jest czas przyjazdu, a drugim parametrem liczba przesiadek. W trybie przesiadek kolejność jest odwrotna. Dzięki temu przy identycznym czasie przyjazdu algorytm automatycznie wybierze połączenie z mniejszą liczbą przesiadek, co znacząco poprawia jakość zwracanych tras.

In [ ]:
```
import heapq
import math
```

```python
class AStar:
    def __init__(self, builder, metadata):
        # Initialize graph and station data
        self.graph = builder.build_graph()
        self.metadata = metadata

    def haversine(self, stop1, stop2):
        # Calculate distance in km between two sets of coordinates
        s1 = self.metadata[stop1]
        s2 = self.metadata[stop2]

        R = 6371
        d_lat = math.radians(s2.lat - s1.lat)
        d_lon = math.radians(s2.lon - s1.lon)
        a = (math.sin(d_lat / 2)**2 +
             math.cos(math.radians(s1.lat)) * math.cos(math.radians(s2.lat)) * m
        return R * (2 * math.atan2(math.sqrt(a), math.sqrt(1 - a)))

    def get_heuristic(self, current_stop, goal_stop, mode):
        # Estimated cost to reach the destination
        if mode == 'p':
            # For transfers: check if stations share at least one line
            current_lines = self.metadata[current_stop].lines
            goal_lines = self.metadata[goal_stop].lines
            if not (current_lines & goal_lines):
                return 1 # At least one transfer needed
            return 0

        # For time: straight line distance / max train speed
        dist = self.haversine(current_stop, goal_stop)
        v_max_km_s = 200 / 3600 # 200 km/h
        return dist / v_max_km_s

    def shortest_path(self, start_station, goal_station, start_time, mode="t", t
        open_list = [] # Nodes to check
        closed_list = set() # Already checked nodes

        g_costs = {} # Best known cost to reach a state
        parent = {} # Track the path
        parent_edge = {}
        entry_count = 0 # Tie-breaker for the priority queue

        # State is (station_id, trip_id) to handle transfers correctly
        start_state = (start_station, None)
        start_g = (start_time, 0) if mode == "t" else (0, start_time)
        h_val = self.get_heuristic(start_station, goal_station, mode)

        # f = actual cost (g) + estimated cost to goal (h)
        start_f = (start_g[0] + h_val, start_g[1])

        heapq.heappush(open_list, (start_f, entry_count, start_g, start_time, st
        g_costs[start_state] = start_g

        while open_list:
            # Pick the most promising node
            f, _, g, current_time, u, current_trip = heapq.heappop(open_list)
            current_state = (u, current_trip)

            if u == goal_station:
```

```
                return self._reconstruct_path(parent, parent_edge, start_station

            if current_state in closed_list:
                continue

            closed_list.add(current_state)

            # Check all outgoing connections
            for edge in self.graph.get(u, []):
                is_transfer = 1 if current_trip is not None and current_trip !=
                required_dep = current_time + (transfer_buffer if is_transfer el

                # Skip if train departs too early
                if edge.departure < required_dep:
                    continue

                if mode == "t":
                    new_g = (edge.arrival, g[1] + is_transfer)
                else:
                    new_g = (g[0] + is_transfer, edge.arrival)

                v = edge.to_stop
                neighbor_state = (v, edge.trip_id)

                # Update if we found a better way to this state
                if neighbor_state not in g_costs or new_g < g_costs[neighbor_sta
                    g_costs[neighbor_state] = new_g
                    h = self.get_heuristic(v, goal_station, mode)

                    new_f = (new_g[0] + h, new_g[1])

                    parent[neighbor_state] = current_state
                    parent_edge[neighbor_state] = edge

                    # Re-check if it was already "finished" but now has a better
                    if neighbor_state in closed_list:
                        closed_list.remove(neighbor_state)

                    entry_count += 1
                    heapq.heappush(open_list, (new_f, entry_count, new_g, edge.a

        return None, float("inf")

    def _reconstruct_path(self, parent, parent_edge, start_station, goal_station
        # Backtrack from goal to start using the parent dictionary
        path = []

        # Find which specific trip reached the goal
        current_state = None
        for state in parent_edge:
            if state[0] == goal_station:
                current_state = state
                break

        if not current_state:
            return None

        while current_state in parent_edge:
            edge = parent_edge[current_state]
            path.append((current_state[0], edge))
```

```
            current_state = parent[current_state]

        path.append((start_station, None))
        path.reverse()
        return path
```

Podsumowanie i wnioski

Realizacja projektu pozwoliła na głębokie zrozumienie problemu wyszukiwania ścieżek w dynamicznych grafach czasowych. W systemach takich jak sieć kolejowa, klasyczne podejście algorytmów wymaga poważnych modyfikacji. Wyzwaniem nie była sama implementacja Dijkstry czy A*, ale dostosowanie ich do dynamicznej struktury danych. Najbardziej czasochłonnym etapem projektu było samo przetwarzanie danych GTFS i budowanie grafu, na którym operują algorytmy, ponieważ wymagało to dokładnego i głębokiego zrozumienia problemu.

Z przeprowadzonych przeze mnie testów wynika, że zarówno Dijkstra, jak i A* są w stanie znaleźć w miarę optymalne rozwiązania. Jednak A* robi to znacznie szybciej, ponieważ zamiast eksplorować wszystie możliwe ścieżki, skupia się na tych, które prowadzą w stronę celu. W przypadku Dijkstry, liczba odwiedzonych stanów rośnie bardzo szybko, niekoniecznie dając lepsze rozwiązanie.

W trakcie pracy nad kodem korzystałam głównie z materiałów załączonych w instrukcji. Dodatkowo, korzystałam również z LLM (Gemini) w celu debugowania błędów, weryfikacji poprawności implementacji oraz w celu napisania funkcji testujących kod i formatujących wyniki.

Ostatecznie projekt poazał, że w starciu z rzeczywistymi danymi nawet teoretycznie optymalne algorytmy wymagają szeregu modyfikacji, aby zwracane przez nie wyniki były użyteczne i optymalne.